

# Week 1 Tutorial - Introduction to OR

Before completing this tutorial you should complete the following **Basic Vector and Matrix Operations in NumPy**.

*You also might want to have it open to refer to while trying to complete this tutorial.*

The following exercises are aimed at graphing inequalities and visualising systems of equations using Python, there are many programs (software) which can do this, however we will focus on Python throughout the course for consistency.

A good understanding of vectors and matrices will really prove beneficial when studying topics in Operational Research such as Linear Programming, Integer Programming and Networks to name a few.

***p.s if you want to learn some latex, you can double-click on the cells to see the latex code.***

## Exercise 1

Find the determinant of the following matrices by hand:

a.

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

b.

$$\begin{pmatrix} 1 & -2 \\ 3 & 4 \end{pmatrix}$$

c.

$$\begin{pmatrix} 2 & 6 \\ 4 & 12 \end{pmatrix}$$

## ▼ Exercise 2

Use Python to find the determinants and check your answers to Exercise 1.

I have provided a runnable cell for you. Just click in the cell and press Shift+Enter to run the code.

We will be using NumPy which is a library that supports vectors/matrices and their operations. In NumPy they are called arrays.

```
1 import numpy as np
2
3 # Place your answer here for Exercise 2
4 print("Write you code here for Exercise 2!")

Write you code here!
```

## ▼ Exercise 3

a) Solve the following system of equations by hand using elimination or substitution.

$$3x + y = 11$$

$$x + 4y = 11$$

b) Solve using matrix form

c) Use Python to solve this system.

```
1 import numpy as np
2
3 # Place your answer here for Exercise 3
4 print("Write you code here for Exercise 3!")
```

## ▼ Exercise 4

Produce a plot of your solution using Python

Amend the code below.

```
1 import matplotlib.pyplot as plt
2
3 # Here it is in python!
```

```

4
5 # This is a square matrix with 2 rows and 2 columns (2,2).
6 A = np.array([[1,2],
7               [3,4]
8               ])
9
10 b = np.array([1,3])
11
12 # We can solve a system of linear equations using the np.linalg.det() function.
13 x = np.dot(np.linalg.inv(A), b)
14
15 X = np.linspace(x[0] - 5, x[0] + 5, num=100)
16 Y1 = (b[0] - A[0,0]*X)/A[0,1]
17 Y2 = (b[1] - A[1,0]*X)/A[1,1]
18
19 plt.plot(X,Y1)
20 plt.plot(X,Y2)
21 plt.plot(x[0], x[1], "ko")
22 plt.xlabel("$x$")
23 plt.ylabel("$y$")
24
25 plt.show()

```

## Exercise 5

Produce a plot of the solution on [desmos.com](https://www.desmos.com)

## Exercise 6

Graph the following system of inequalities by hand. Shade the feasible region.

$$\begin{aligned}
 x + y &\leq 2 \\
 x + 3y &\leq 3 \\
 x &\geq 0 \\
 y &\geq 0
 \end{aligned}$$

Confirm this by plotting this on [desmos.com](https://www.desmos.com)

## ▼ Exercise 7

Solve the following system using Python.

$$\begin{aligned}
 2x + 3y - 2z &= 3 \\
 x - 2y + 4z &= 0 \\
 2x - 3y + z &= 1
 \end{aligned}$$

You will need to work out **A** and **b**.

$$\mathbf{Ax} = \mathbf{b}$$

$$\begin{pmatrix} * & * & * \\ * & * & * \\ * & * & * \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} * \\ * \\ * \end{pmatrix}$$

```

1 import numpy as np
2
3 # Place your answer here for Exercise 7
4 print("Write you code here for Exercise 7!")

```

Write you code here for Exercise 7!

## ▼ Exercise 8

Visualise your solution by amending the following code.

```

1 import matplotlib.pyplot as plt
2 from mpl_toolkits.mplot3d import Axes3D
3
4 A = np.array([
5     [ 2,  3,  1],
6     [-1,  6,  1],
7     [-1, -3,  1]
8 ])

```

```

9
10 b = np.array([4, 2, -4])
11
12 # We can solve a system of linear equations using the np.linalg.det() function.
13 x = np.dot(np.linalg.inv(A), b)
14
15 X = np.linspace(-10, 10, num=20)
16 Y = np.linspace(-10, 10, num=20)
17 xv, yv = np.meshgrid(X, Y)
18
19 z1 = (b[0] - A[0,0]*xv - A[0,1]*yv)/A[0,2];
20 z2 = (b[1] - A[1,0]*xv - A[1,1]*yv)/A[1,2];
21 z3 = (b[2] - A[2,0]*xv - A[2,1]*yv)/A[2,2];
22
23
24 fig, ax = plt.subplots(subplot_kw={"projection": "3d"})
25
26 # https://matplotlib.org/stable/api/\_as\_gen/mpl\_toolkits.mplot3d.axes3d.Axes3D.view\_init.html
27 ax.view_init(0, 0) # use this to manually rotate your plot
28 ax.plot_surface(xv,yv,z1)
29 ax.plot_surface(xv,yv,z2)
30 ax.plot_surface(xv,yv,z3)
31
32 plt.show()
33

```

I have also provided an interactive version of the plot.

```

1 from ipywidgets import interact, fixed
2 # Importing mplot3d toolset
3 from mpl_toolkits.mplot3d import Axes3D
4
5
6
7 @interact(elev=(-90, 90), azimuth=(-180, 180))
8 def plot_3D(elev=30, azimuth=30):
9     A = np.array([
10         [ 2,  3,  1],
11         [-1,  6,  1],
12         [-1, -3,  1]
13     ])
14
15     b = np.array([4, 2, -4])
16     # We can solve a system of linear equations using the np.linalg.det() function.
17     x = np.dot(np.linalg.inv(A), b)
18
19     # Data for three-dimensional scattered points
20     X = np.linspace(-10, 10, num=20)
21     Y = np.linspace(-10, 10, num=20)
22     xv, yv = np.meshgrid(X, Y)
23
24     z1 = (b[0] - A[0,0]*xv - A[0,1]*yv)/A[0,2];
25     z2 = (b[1] - A[1,0]*xv - A[1,1]*yv)/A[1,2];
26     z3 = (b[2] - A[2,0]*xv - A[2,1]*yv)/A[2,2];
27
28
29     fig, ax = plt.subplots(subplot_kw={"projection": "3d"})
30
31     # https://matplotlib.org/stable/api/\_as\_gen/mpl\_toolkits.mplot3d.axes3d.Axes3D.view\_init.html
32     ax.view_init(elev, azimuth) # use this to manually rotate your plot
33     ax.scatter(x[0], x[1], x[2], c='k', s=20)
34     ax.plot_surface(xv,yv,z1, alpha=0.3)
35     ax.plot_surface(xv,yv,z2, alpha=0.3)
36     ax.plot_surface(xv,yv,z3, alpha=0.3)
37
38
39     plt.show()

```

